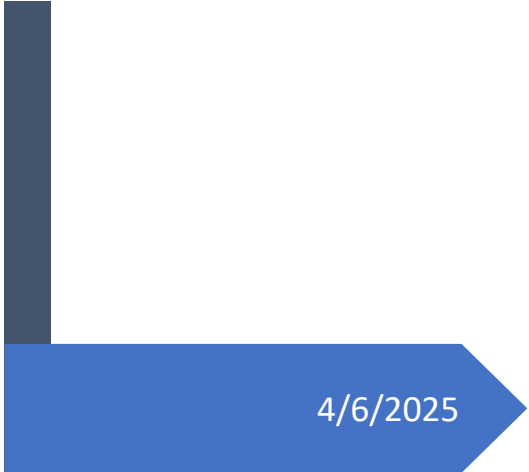
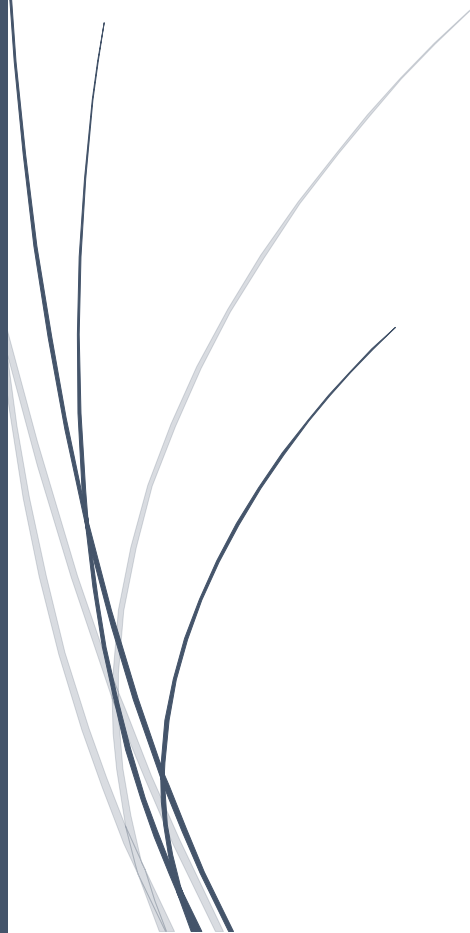


A dark blue vertical bar on the left side of the page. A blue arrow points to the right from this bar, containing the date 4/6/2025.

4/6/2025

Αναφορά

Πρόβλεψη Παραπομπών με Χρήση Link
Prediction σε Δίκτυα Επιστημονικών Άρθρων

Several thin, curved lines in dark blue and light grey originate from the bottom left corner and curve upwards and to the right.

Μουσελίμη Αμαλία-Γεωργία 5074
Σακελλαρίου Ιωάννα 5125

1. Εισαγωγή

Η εργασία αυτή ασχολείται με την πρόβλεψη ετεροαναφορών (citations) μεταξύ επιστημονικών άρθρων. Το πρόβλημα προσεγγίζεται ως πρόβλημα δυαδικής ταξινόμησης. Δοθέντος ενός ζεύγους άρθρων (A, B), προβλέπουμε αν υπάρχει παραπομπή από το A στο B.

Η εργασία αφορούσε την πρόβλεψη της πιθανότητας με την οποία ένα επιστημονικό άρθρο A θα παραπέμψει σε ένα άλλο άρθρο B, αξιοποιώντας πληθώρα πληροφοριών από:

- Το κείμενο των περιλήψεων (abstracts),
- Τους συγγραφείς των άρθρων,
- Τη γραφική δομή του citation network.

Ο στόχος ήταν η ανάπτυξη ενός μοντέλου ταξινόμησης που λαμβάνει ως είσοδο δύο paper IDs και επιστρέφει μια πιθανότητα παραπομπής. Η αξιολόγηση γινόταν μέσω της μετρικής log loss, τόσο στο validation set όσο και στο test set που αξιολογείται μέσω της Kaggle πλατφόρμας.

Για την επίλυση του προβλήματος υλοποιήσαμε:

- Μηχανική εξαγωγή χαρακτηριστικών από γράφους και κείμενο,
- Τεχνικές επεξεργασίας φυσικής γλώσσας (Sentence-BERT, TF-IDF),
- Δημιουργία θετικών και αρνητικών δειγμάτων (data augmentation),

2. Περιγραφή Δεδομένων

Για την επίλυση του προβλήματος αξιοποιήθηκαν τρία βασικά αρχεία εισόδου:

- abstracts.txt: Περιέχει τις περιλήψεις (abstracts) των επιστημονικών εργασιών. Κάθε γραμμή έχει τη μορφή: paper_id |--| abstract_text.
- authors.txt: Παρέχει πληροφορία για τους συγγραφείς κάθε άρθρου. Κάθε γραμμή έχει τη μορφή: paper_id |--| author1,author2,....
- edgelist.txt: Περιέχει τις θετικές ακμές μεταξύ papers σε μορφή source, target.

Επιπλέον, δόθηκε αρχείο test.txt με paper ζεύγη (source, target) για τα οποία έπρεπε να γίνει πρόβλεψη.

Παρατηρήσεις:

- Το γράφημα παραπομπών είναι κατευθυνόμενο (directed) και αραιό.
- Υπάρχει μεγάλη ποικιλία στη θεματολογία των περιλήψεων, γεγονός που αυξάνει την πολυπλοκότητα της εξαγωγής σημασιολογικών χαρακτηριστικών.

- Στο σύνολο δεδομένων δίνονται μόνο θετικά παραδείγματα, τα αρνητικά δημιουργούνται τεχνητά (negative sampling).

3. Επεξεργασία Δεδομένων

Σε αυτή την ενότητα περιγράφουμε βήμα προς βήμα την προσέγγισή μας για την επίλυση του προβλήματος ταξινόμησης (classification). Το ζητούμενο είναι να προβλέψουμε εάν ένα paper A παραπέμπει (cites) σε paper B. Η συνολική διαδικασία αποτελείται από τα εξής στάδια:

3.1 Καθαρισμός και Φόρτωση Δεδομένων

Τα δεδομένα φορτώνονται από τρία βασικά αρχεία:

- abstracts.txt
- authors.txt.
- edgelist.txt

Δημιουργήθηκε ένα κατευθυνόμενο γράφημα (directed graph) με χρήση της βιβλιοθήκης NetworkX για να μοντελοποιηθεί το citation network.

3.2 Δημιουργία Αρνητικών Δειγμάτων

Εκτός από τα θετικά ζεύγη (papers που όντως παραπέμπουν), δημιουργήθηκαν αρνητικά ζεύγη (που δεν παραπέμπουν).

Υλοποιήθηκε παραγωγή τυχαίων αρνητικών δειγμάτων, επιλογή ζευγών χωρίς ακμή. Αξίζει να σημειωθεί ότι προσπαθήσαμε να υλοποιήσουμε και “hard negatives”, όπου επιλέγονται ζεύγη με υψηλή ομοιότητα abstracts αλλά χωρίς ακμή, ώστε το μοντέλο να εκπαιδεύεται σε πιο δύσκολες περιπτώσεις, αλλά δεν καταφέραμε να το τρέξουμε διότι ήθελε πάρα πολλές ώρες.

3.3 Εξαγωγή Χαρακτηριστικών (Feature Engineering)

Για κάθε ζεύγος (source, target) εξάγονται τα εξής χαρακτηριστικά:

- Ομοιότητα abstracts: Υπολογίζεται με Cosine Similarity σε ενσωματώσεις κειμένου.
- Κοινοί συγγραφείς: Αριθμός κοινών συγγραφέων μεταξύ των δύο papers.
- Κοινοί γείτονες (graph-based): Κοινές εξερχόμενες και εισερχόμενες ακμές στον γράφο.
- Προϊόν βαθμών (degree product): Πολλαπλασιασμός των βαθμών (degree) των κόμβων.

Έγινε ομαδοποίηση των χαρακτηριστικών σε παρτίδες των 100.000 για εξοικονόμηση RAM.

Τα αποτελέσματα αποθηκεύτηκαν σε .npy αρχεία στο στάδιο ενδιάμεσης επεξεργασίας.

4. Feature Engineering

Για την περιγραφή της σχέσης μεταξύ δύο άρθρων, κατασκευάστηκε ένα σύνολο χαρακτηριστικών που συνδυάζουν σημασιολογική, δομική και μεταδεδομένη πληροφορία.

Χρήση Sentence-BERT (all-MiniLM-L6-v2):

- Επιλέχθηκε επειδή παρέχει συμπαγείς και αποδοτικές ενσωματώσεις (embeddings) προτάσεων με νόημα, κατάλληλες για semantic similarity tasks.
- Κάθε abstract μετατράπηκε σε διάνυσμα 384 διαστάσεων.
- Υπολογίστηκε cosine similarity μεταξύ των embeddings του paper A και του B, για να μετρηθεί η εννοιολογική εγγύτητα.

Γραφικά & Συγγραφικά Χαρακτηριστικά:

- Κοινοί συγγραφείς: Αν δύο άρθρα έχουν κοινούς συγγραφείς, αυξάνεται η πιθανότητα παραπομπής. Το χαρακτηριστικό αυτό καταμετρά το πλήθος τους.
- Κοινοί γείτονες (Common Neighbors): Μετρά τον αριθμό κόμβων που είναι κοινά συνδεδεμένοι και με τους δύο κόμβους.
- Προϊόν βαθμών (Preferential Attachment): Το γινόμενο των βαθμών (degree) των δύο κόμβων. Με βάση τη θεωρία προτιμησιακής σύνδεσης, οι κόμβοι με μεγάλο βαθμό είναι πιο πιθανό να συνδεθούν.

Σύνολο χαρακτηριστικών ανά ζεύγος: 772 (384 για το A + 384 για το B + 4 engineered features)

Βελτιστοποίηση Υποδομής:

- StandardScaler: Εφαρμόστηκε για κανονικοποίηση συνεχών χαρακτηριστικών, βελτιώνοντας τη σταθερότητα των μοντέλων.
- Caching: Οι ενσωματώσεις και τα τελικά χαρακτηριστικά αποθηκεύτηκαν σε .npy αρχεία ώστε να αποφευχθεί επανυπολογισμός.
- Batch extraction: Για αποφυγή εξάντλησης RAM, όλα τα δεδομένα επεξεργάζονταν σε παρτίδες (π.χ. των 100.000 ζευγών), διευκολύνοντας το scalable pipeline.

5. Αρχιτεκτονική Μοντέλων

Χρησιμοποιήσαμε το **Feedforward Νευρωνικό Δίκτυο (PyTorch)**. Επιλέχθηκε ως baseline deep learning μοντέλο λόγω της δυνατότητας εκμάθησης μη γραμμικών σχέσεων μεταξύ σύνθετων χαρακτηριστικών.

- **Είσοδος:** 772 χαρακτηριστικά (384 + 384 από Sentence-BERT, 4 engineered features).
- **Κρυφά Επίπεδα:**
 - 1ο επίπεδο: 256 νευρώνες - αρχική μετασχηματιστική προβολή.

- 2ο επίπεδο: 128 νευρώνες - ενδιάμεση αναπαράσταση.
- 3ο επίπεδο: 64 νευρώνες - σύνοψη για την έξοδο.
- **Ενεργοποίηση (ReLU):** $\max(0, x)$, επιτρέπει μη γραμμική μοντελοποίηση χωρίς vanishing gradient.
- **Dropout(0.2):** Τακτική regularization που απενεργοποιεί τυχαία 20% των νευρώνων σε κάθε forward pass για αποφυγή overfitting.
- **Έξοδος:** 1 νευρώνας με **Sigmoid** ενεργοποίηση - κατάλληλη για δυαδική ταξινόμηση.

Αξίζει να σημειωθεί ότι δοκιμάσαμε επιπλέον τα μοντέλα αρχιτεκτονικής LightGBM, XGBoost και Optuna αλλά προτιμήσαμε το PyTorch νευρωνικό δίκτυο γιατί παρατηρήσαμε καλύτερα αποτελέσματα.

6. Εκπαίδευση του Μοντέλου

Η εκπαίδευση του νευρωνικού δικτύου πραγματοποιείται μέσω της συνάρτησης `train_model`. Σε αυτό το στάδιο, χρησιμοποιήθηκε το πλήρως συνδεδεμένο νευρωνικό δίκτυο που αναφέρθηκε παραπάνω, με στόχο την ελαχιστοποίηση του σφάλματος πρόβλεψης για το αν υπάρχει παραπομπή μεταξύ δύο άρθρων.

Διαδικασία εκπαίδευσης:

1. **Ορισμός Συσκευής Υπολογισμού:** Το μοντέλο μεταφέρεται αυτόματα σε GPU (CUDA) αν υπάρχει διαθέσιμη, διαφορετικά χρησιμοποιείται η CPU.
2. **Βελτιστοποιητής:** Χρησιμοποιείται ο Adam optimizer με ρυθμό εκμάθησης (learning rate) ίσο με 0.001. Ο Adam επιλέγεται γιατί συνδυάζει momentum και προσαρμογή βημάτων, επιταχύνοντας τη σύγκλιση.
3. **Συνάρτηση Κόστους:** Η Binary Cross Entropy Loss (BCELoss) μετρά την απόκλιση μεταξύ της προβλεπόμενης πιθανότητας και της πραγματικής ετικέτας (0 ή 1). Είναι κατάλληλη για δυαδική ταξινόμηση.
4. **Διαχείριση Δεδομένων:** Τα δεδομένα εκπαίδευσης και επικύρωσης (validation) οργανώνονται σε batches μέσω PyTorch DataLoaders, με μέγεθος παρτίδας (batch size) ίσο με 64.
5. **Εκπαίδευση ανά Epoch:**
 - Για κάθε epoch:
 - Το μοντέλο τίθεται σε κατάσταση εκπαίδευσης (`model.train()`).
 - Για κάθε batch, υπολογίζεται η πρόβλεψη, το loss, και εκτελείται ο βρόχος backpropagation για ενημέρωση των βαρών.
 - Στο τέλος κάθε epoch, το μοντέλο τίθεται σε κατάσταση αξιολόγησης (`model.eval()`) και υπολογίζεται το μέσο validation loss για το validation set, ώστε να παρακολουθείται η γενίκευση και να ελέγχεται το overfitting.

6. **Επαναλήψεις:** Το μοντέλο εκπαιδεύεται για 20 επαναλήψεις (epochs), αριθμός που επιλέχθηκε ώστε να επιτυγχάνεται ικανοποιητική σύγκλιση χωρίς υπερπροσαρμογή.

Η συνάρτηση επιστρέφει το πλήρως εκπαιδευμένο μοντέλο, έτοιμο για πρόβλεψη στις νέες ακμές του test set.

7. Πειραματικά Αποτελέσματα

Πραγματοποιήθηκαν εκτεταμένοι πειραματισμοί με διάφορους συνδυασμούς χαρακτηριστικών και μοντέλων για την εξαγωγή του βέλτιστου pipeline. Η τελική επιλογή βασίστηκε σε απόδοση, ταχύτητα και δυνατότητα γενίκευσης.

Log Loss στο validation set: 0.13999

Σύγκριση μοντέλων:

- **TF-IDF + LightGBM:**
 - Πολύ γρήγορη προσέγγιση (εκπαίδευση < 15 λεπτά)
 - Επίδοση: ~0.194 log loss
 - Ιδανική για χαμηλό resource cost ή γρήγορη ανάπτυξη
 - Δεν έχει τόσο καλό αποτέλεσμα
- **SBERT + XGBoost:**
 - Συνδυάζεται σημασιολογική πληροφορία με δομικά χαρακτηριστικά
 - Επίδοση: ~0.170 log loss
 - Ακριβέστερο αλλά έχει μεγάλο υπολογιστικό κόστος (>2 ώρες)
- **SBERT + XGBoost + Hard Negatives:**
 - Δύσκολα παραδείγματα βελτιώνουν την ικανότητα γενίκευσης του μοντέλου
 - Επίδοση: ~0.165 log loss (βελτίωση χωρίς αλλαγή μοντέλου)

8. Τεχνικές Βελτιστοποίησης

Λόγω του μεγέθους του dataset, εφαρμόστηκαν τεχνικές για αύξηση της αποδοτικότητας και αποφυγή υπερφόρτωσης υπολογιστικών πόρων:

- **Caching με .npy αρχεία:**

Τα Sentence-BERT embeddings και τα τελικά χαρακτηριστικά αποθηκεύτηκαν τοπικά και φορτώνονταν on demand.

- **Batch Processing:**

Οι υπολογισμοί χωρίστηκαν σε batches των 100.000 ζευγών. Έτσι μειώθηκε η χρήση μνήμης και βελτιώθηκε η ταχύτητα εκτέλεσης του προγράμματος.

- **Εγγραφή προσωρινών αρχείων στον δίσκο:**

Αντί για χρήση RAM, η ενδιάμεση αποθήκευση έγινε στον δίσκο (SSD) σε μορφή NumPy arrays. Έγινε χρήση pickle για ταχύτητα αποθήκευσης/φόρτωσης.

9. Συμπεράσματα

- Η ποιότητα των χαρακτηριστικών που εξάγονται από τις περιλήψεις παίζει καθοριστικό ρόλο στην επίδοση του συστήματος. Τα embeddings από Sentence-BERT ξεπέρασαν το TF-IDF σε ακρίβεια και σημασιολογική έκφραση, αν και απαιτούν περισσότερους υπολογιστικούς πόρους.
- Οι γραφικές πληροφορίες (neighbors, degree) ενισχύουν την ακρίβεια.
- Η δημιουργία "δύσκολων" αρνητικών παραδειγμάτων (hard negatives) συνέβαλε καθοριστικά στη βελτίωση του μοντέλου, αλλά επιβαρύνει σημαντικά το μοντέλο σε υπολογιστικό κόστος .
- Το PyTorch αποτελεί την καλύτερη και πιο ισορροπημένη επιλογή για να αξιοποιηθεί πλήρως η πληροφορία των embedding και του γράφου, καθώς προσφέρει καλή απόδοση, χαμηλό log loss, με σταθερή γενίκευση συγκριτικά με LightGBM και XGBoost που αποδίδουν καλύτερα σε λιγότερες διαστάσεις.